Documento de Persistencia — SmartLogix

Generado: 2026-06-12T02:32:33.036Z

Resumen general

- Motor de base de datos: PostgreSQL
- ORM/mapper: Sequelize (Node.js) — equivalente a JPA en Java
- Patrón: cada microservicio gestiona su propia BD (DB per service)
- Conexión: variables de entorno `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT`

inventory-service

Modelos / Entidades

Modelo	Tabla	Campos principales
Product	products	id, sku, name, description, category, price
Stock	stocks	id, productId, warehouseId, quantity, reservedQuantity
Warehouse	warehouses	id, name, location, capacity, isActive
index	—	

Fragmentos de código (primeras líneas)

Product.js

```
const { DataTypes } = require('sequelize');

// ── Entidad Producto (@Entity equivalente) ──────────────────────────────
module.exports = (sequelize) => sequelize.define('Product', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  sku: {
    type: DataTypes.STRING(50),
    allowNull: false,
    unique: true,
  },
  name: {
    type: DataTypes.STRING(200),
    allowNull: false,
  },
  description: {
    type: DataTypes.TEXT,
  },
  category: {
    type: DataTypes.STRING(100),
  },
  price: {
    type: DataTypes.DECIMAL(10, 2),
    allowNull: false,
    defaultValue: 0,
  },
```

```
  unit: {
    type: DataTypes.STRING(20),
    defaultValue: 'unidad',
  },
  minStock: {
    type: DataTypes.INTEGER,
    defaultValue: 5,
    field: 'min_stock',
  },
  isActive: {
    type: DataTypes.BOOLEAN,
    defaultValue: true,
    field: 'is_active',
  },
}, {
  tableName: 'products',
  timestamps: true,
  underscored: true,
});
```

Stock.js

```
const { DataTypes } = require('sequelize');

module.exports = (sequelize) => sequelize.define('Stock', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  productId: {
    type: DataTypes.UUID,
    allowNull: false,
    field: 'product_id',
  },
  warehouseId: {
    type: DataTypes.UUID,
    allowNull: false,
    field: 'warehouse_id',
  },
  quantity: {
    type: DataTypes.INTEGER,
    allowNull: false,
    defaultValue: 0,
  },
  reservedQuantity: {
    type: DataTypes.INTEGER,
    defaultValue: 0,
    field: 'reserved_quantity',
  },
}, {
  tableName: 'stocks',
  timestamps: true,
  underscored: true,
});
```

Warehouse.js

```
const { DataTypes } = require('sequelize');

module.exports = (sequelize) => sequelize.define('Warehouse', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  name: {
    type: DataTypes.STRING(150),
    allowNull: false,
  },
```

```
  location: {
    type: DataTypes.STRING(250),
  },
  capacity: {
    type: DataTypes.INTEGER,
    defaultValue: 1000,
  },
  isActive: {
    type: DataTypes.BOOLEAN,
    defaultValue: true,
    field: 'is_active',
  },
}, {
  tableName: 'warehouses',
  timestamps: true,
  underscored: true,
});
```

index.js

```
const { Sequelize } = require('sequelize');

// ── Conexión a PostgreSQL (equivalente a JPA DataSource) ──────────────────
const sequelize = new Sequelize(
  process.env.DB_NAME || 'inventorydb',
  process.env.DB_USER || 'smartlogix',
  process.env.DB_PASSWORD || 'smartlogix123',
  {
    host: process.env.DB_HOST || 'localhost',
    port: process.env.DB_PORT || 5432,
    dialect: 'postgres',
    logging: false,
    pool: { max: 10, min: 0, acquire: 30000, idle: 10000 },
  }
);

// ── Entidades (equivalente a @Entity en JPA) ──────────────────────────────
const Product = require('./Product')(sequelize);
const Warehouse = require('./Warehouse')(sequelize);
const Stock = require('./Stock')(sequelize);

// ── Relaciones ────────────────────────────────────────────────────────────
Product.hasMany(Stock, { foreignKey: 'productId', as: 'stocks' });
Warehouse.hasMany(Stock, { foreignKey: 'warehouseId', as: 'stocks' });
Stock.belongsTo(Product, { foreignKey: 'productId', as: 'product' });
Stock.belongsTo(Warehouse, { foreignKey: 'warehouseId', as: 'warehouse' });

module.exports = { sequelize, Product, Warehouse, Stock };
```

Relaciones y notas

Relaciones: `Product.hasMany(Stock)`, `Warehouse.hasMany(Stock)`, `Stock.belongsTo(Product)`.

Tablas: `products`, `stocks`, `warehouses`.

orders-service

Modelos / Entidades

Modelo	Tabla	Campos principales
Order	orders	id, orderNumber, customerId, customerName, customerEmail, status
OrderItem	order_items	id, orderId, productId, productName, quantity, unitPrice

Modelo	Tabla	Campos principales
index	—	

Fragmentos de código (primeras líneas)

Order.js

```
const { DataTypes } = require('sequelize');
const { randomInt } = require('crypto');

// Estados del pedido: flujo de trazabilidad completo
const ORDER_STATUS = ['PENDING', 'VALIDATED', 'APPROVED', 'PROCESSING', 'SHIPPED', 'DELIVERED

module.exports = (sequelize) => sequelize.define('Order', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  orderNumber: {
    type: DataTypes.STRING(20),
    unique: true,
    field: 'order_number',
  },
  customerId: {
    type: DataTypes.UUID,
    allowNull: false,
    field: 'customer_id',
  },
  customerName: {
    type: DataTypes.STRING(200),
    field: 'customer_name',
  },
  customerEmail: {
    type: DataTypes.STRING(200),
    field: 'customer_email',
  },
  status: {
    type: DataTypes.ENUM(...ORDER_STATUS),
    defaultValue: 'PENDING',
  },
  totalAmount: {
    type: DataTypes.DECIMAL(12, 2),
    defaultValue: 0,
    field: 'total_amount',
  },
  shippingAddress: {
    type: DataTypes.TEXT,
    field: 'shipping_address',
  },
  paymentStatus: {
    type: DataTypes.ENUM('PENDING', 'PAID', 'FAILED', 'REFUNDED'),
    defaultValue: 'PENDING',
    field: 'payment_status',
  },
  paymentId: {
    type: DataTypes.STRING(200),
    field: 'payment_id',
  },
  notes: {
    type: DataTypes.TEXT,
  },
}, {
  tableName: 'orders',
  timestamps: true,
  underscored: true,
  hooks: {
    beforeCreate: (order) => {
      // Genera número de pedido automático
```

```
      if (!order.orderNumber) {
        const date = new Date();
        const num = randomInt(1000, 10000);
        order.orderNumber = `ORD-${date.getFullYear()}${(date.getMonth()+1).toString().padSta
      }
    },
  },
});
```

OrderItem.js

```
const { DataTypes } = require('sequelize');

module.exports = (sequelize) => sequelize.define('OrderItem', {
  id: {
    type: DataTypes.UUID,
    defaultValue: DataTypes.UUIDV4,
    primaryKey: true,
  },
  orderId: {
    type: DataTypes.UUID,
    allowNull: false,
    field: 'order_id',
  },
  productId: {
    type: DataTypes.UUID,
    allowNull: false,
    field: 'product_id',
  },
  productName: {
    type: DataTypes.STRING(200),
    field: 'product_name',
  },
  quantity: {
    type: DataTypes.INTEGER,
    allowNull: false,
    defaultValue: 1,
  },
  unitPrice: {
    type: DataTypes.DECIMAL(10, 2),
    allowNull: false,
    field: 'unit_price',
  },
  subtotal: {
    type: DataTypes.DECIMAL(12, 2),
    allowNull: false,
  },
}, {
  tableName: 'order_items',
  timestamps: true,
  underscored: true,
});
```

index.js

```
const { Sequelize } = require('sequelize');

const sequelize = new Sequelize(
  process.env.DB_NAME || 'ordersdb',
  process.env.DB_USER || 'smartlogix',
  process.env.DB_PASSWORD || 'smartlogix123',
  {
    host: process.env.DB_HOST || 'localhost',
    port: process.env.DB_PORT || 5432,
    dialect: 'postgres',
    logging: false,
    pool: { max: 10, min: 0, acquire: 30000, idle: 10000 },
  }
);
```

```
const Order = require('./Order')(sequelize);
const OrderItem = require('./OrderItem')(sequelize);

// Relaciones
Order.hasMany(OrderItem, { foreignKey: 'orderId', as: 'items' });
OrderItem.belongsTo(Order, { foreignKey: 'orderId', as: 'order' });

module.exports = { sequelize, Order, OrderItem };
```

Relaciones y notas

Relaciones: `Order.hasMany(OrderItem)`, `OrderItem.belongsTo(Order)`.

Tablas: `orders`, `order_items`.

shipping-service

Modelos / Entidades

Modelo	Tabla	Campos principales
index	shipments	id, orderId, productId, warehouseId, trackingNumber, carrier

Fragmentos de código (primeras líneas)

index.js

```
const { Sequelize, DataTypes } = require('sequelize');
const { randomInt } = require('crypto');

const sequelize = new Sequelize(
  process.env.DB_NAME || 'shippingdb',
  process.env.DB_USER || 'smartlogix',
  process.env.DB_PASSWORD || 'smartlogix123',
  { host: process.env.DB_HOST || 'localhost', port: process.env.DB_PORT || 5432, dialect: 'po
);

const Shipment = sequelize.define('Shipment', {
  id: { type: DataTypes.UUID, defaultValue: DataTypes.UUIDV4, primaryKey: true },
  orderId: { type: DataTypes.UUID, allowNull: false, field: 'order_id' },
  productId: { type: DataTypes.UUID, allowNull: true, field: 'product_id' },
  warehouseId: { type: DataTypes.UUID, allowNull: true, field: 'warehouse_id' },
  trackingNumber: { type: DataTypes.STRING(50), unique: true, field: 'tracking_number' },
  carrier: { type: DataTypes.STRING(100), defaultValue: 'Starken' },
  status: {
    type: DataTypes.ENUM('PENDING','PICKED_UP','IN_TRANSIT','OUT_FOR_DELIVERY','DELIVERED','F
    defaultValue: 'PENDING',
  },
  originAddress: { type: DataTypes.TEXT, field: 'origin_address' },
  destinationAddress: { type: DataTypes.TEXT, field: 'destination_address' },
  estimatedDelivery: { type: DataTypes.DATE, field: 'estimated_delivery' },
  deliveredAt: { type: DataTypes.DATE, field: 'delivered_at' },
  weight: { type: DataTypes.DECIMAL(8, 2) },
  notes: { type: DataTypes.TEXT },
}, {
  tableName: 'shipments', timestamps: true, underscored: true,
  hooks: {
    beforeCreate: (s) => {
      if (!s.trackingNumber) {
        s.trackingNumber = `SL-${Date.now()}-${randomInt(1000, 10000)}`;
      }
    },
  },
});
```

```
module.exports = { sequelize, Shipment };
```

Relaciones y notas

Modelo principal: `Shipment` con estados y generación automática de `trackingNumber`.

Tabla: `shipments`.

Recomendaciones

- Agregar migraciones con `sequelize-cli` y scripts para inicializar esquemas en CI/CD.
- Encriptar/ocultar credenciales en variables de entorno o usar secretos del orquestador.
- Agregar índices en campos de búsqueda (ej. `sku`, `order_number`, `tracking_number`).
- Agregar pruebas de integración que validen el mapeo ORM > tablas.

Ubicaciones en el repositorio

- inventory-service: src/models/
- orders-service: src/models/
- shipping-service: src/models/